A compact hybrid language model under CPU inference limits

DASE7506 MP1 | Student ID 3036802959 | 27 September 2026

Abstract

This project trains a small causal language model from scratch on the supplied WikiText-2 text. The final predictor combines a five-block Transformer with local QKV convolution, a compact training-derived count model, and two caches restricted to each evaluation window. Development uses validation only. The frozen model achieves **1.4381909206 full-test bits per byte (BPB)**, compared with 2.1012651958 for the original baseline. It satisfies all three measured resource limits. The personal development target of less than 1.4 BPB was not achieved.

Final result	Measured value
Full-test CPU FP32 BPB	1.4381909205827006
Validation BPB used for selection	1.4217038295231355
Training updates / gradient targets	1,200 / 157,286,400
CPU time / peak RAM / inference assets	4.673x baseline / 2.581 GiB / 58.569 MiB

1. Benchmark and evaluation protocol

The fixed protocol is **7506-mp1-wt2-v2**: the supplied train-fitted BPE tokenizer has 2,048 tokens, and evaluation uses independent windows of 256 next-token targets. The scorer sums negative log-base-2 probabilities and divides by the raw UTF-8 byte count. BPB is not word-level perplexity. The complete validation split has 376,599 targets and 1,148,007 bytes; test has 428,405 targets and 1,292,013 bytes. Every target except the first token of each split is scored once, including the final short window.

Only the supplied training text contributes learned weights or statistics. Validation selects architectures, checkpoints and global mixture settings. No external training text or pretrained weights are used. The original evaluator, tokenizer, benchmark files and baseline implementation retain their supplied hashes. Method and checkpoint hashes were recorded before final test evaluation. Three repeated test evaluations reproduce the same score.

This report emphasizes measured comparisons and their limits: the final run has 16 times the baseline gradient-target budget, and component-removal tests do not establish architecture-only causality.

2. Neural model and training

The neural core uses pre-normalized Transformer blocks, tied token/output embeddings, rotary position embeddings [1] and SwiGLU feed-forward layers [2]. A causal depthwise convolution of width three follows the QKV projection and precedes attention. This adopts the local-convolution idea in Primer [3], but keeps SwiGLU rather than reproducing the full Primer architecture.

Setting	Final recipe
Width / depth / attention heads	384 / 5 / 6 (head width 64)
Trainable neural parameters	9,659,520
Feed-forward hidden width	1,024; gated SiLU branch
Residual / embedding / attention dropout	0.20 / 0.10 / 0.05
Context / vocabulary	256 / 2,048
Seed / updates / effective batch	17 / 1,200 / 512
Microbatch / accumulation	64 sequences / 8 microsteps
Precision / training device	BF16 computation, FP32 parameters / Apple MPS

Optimization

Hidden two-dimensional block matrices use a local batched Muon implementation adapted from Keller Jordan [5]: momentum 0.95 and five Newton-Schulz steps. Embeddings, convolution tensors and other parameters use auxiliary AdamW with betas (0.9, 0.95). The peak auxiliary learning rate is 0.004; the Muon group uses a 10x multiplier. Weight decay is 0.3 for auxiliary matrices, 0.03 for Muon matrices and zero for vectors. Global gradient norm is clipped at 1.0.

For zero-indexed update s, the auxiliary rate is 0.004 x min(1, (s+1)/60) x [0.5 + 0.5 cos(pi s/1200)]. Thus the cosine floor is zero and the last actual update is close to, but not exactly, zero. Random length-257 training slices provide 256 targets each. Gradients are averaged across eight microsteps before one optimizer update.

Budget and checkpoint ancestry

The final checkpoint is the unaveraged step-1,200 state of one randomly initialized run. It processes 1,200 x 512 x 256 = **157,286,400 gradient targets**. Intermediate checkpoints support validation and a same-target control; no earlier trained model initializes this run. Training took 8,948.830 recorded seconds; including preparation and scoring, the trainer process took 9,031.021 seconds. BF16 MPS training is separate from ranked CPU FP32 evaluation.

3. A causal hybrid predictor

Let p_N be the neural softmax after temperature scaling, p_G the fixed count expert, p_C the continuous cache and p_S the suffix-copy distribution. The final predictor applies three convex mixtures, followed by log-probability normalization. All mixture settings below were selected on validation before test scoring.

Component	Frozen setting
Neural calibration	Temperature T = 1.2
Train-only count expert	Orders 1-5; discount = 0.5
Count mixture weight	w = 0.1 + 0.2 [max p_G]^2
Continuous cache	Weight 0.025; similarity scale 10
Suffix copy, longest match 2 / 3 / 4-6	Weights 0.10 / 0.20 / 0.25
Unsupported cache or suffix	Use the preceding distribution unchanged

Mixture: p_A = (1-w)p_N + wp_G; p_B = (1-c)p_A + cp_C; p = (1-q)p_B + qp_S. The continuous cache is disabled at the first position; suffix copying is disabled unless a previously observed matching context has at least two tokens.

Training-derived counts

The count expert uses lower-order continuation counts with discounted backoff and raw highest-order counts. Bigram entries are retained; higher orders are generally pruned below count two, except that singleton trigram continuation entries are restored. Unassigned probability mass backs off. Integer keys, pointers and token IDs are packed losslessly by value range; counts stay FP32. The final arrays occupy 22,724,220 bytes. Experimental hashed orders 6-12 are discarded from the submitted predictor.

Within-window adaptation and causality

The continuous cache adapts the hidden-state similarity mechanism in [4]. At position t, it compares the current normalized hidden vector with h_j for j < t and copies token x_(j+1), which is already observed at t. Suffix copying searches for the longest matching context of length at most six; eligible source continuations likewise end no later than t. No cache or hidden state persists across examples, windows or calls.

Future-token perturbation, normalization, repeated-call and batch-independence checks pass for the loaded final checkpoint. CPU inference implements the convolution with equivalent causal shifted products, avoiding expensive padded temporaries. This changes execution cost, not the mathematical predictor. Training continues to use the standard causal convolution path.

4. Development evidence and controls

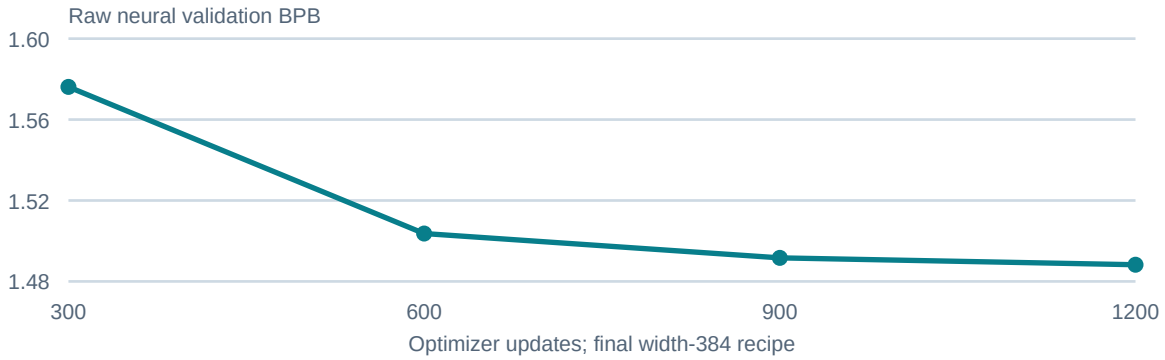
All results in this section are validation BPB. Rows summarize different complete recipes and budgets; they are not controlled single-factor experiments. The final model was chosen using validation quality and resource feasibility, not the test score.

Recipe	Steps	Gradient targets	Validation BPB
Original baseline	1,200	9,830,400	2.071087
Large AdamW neural, batch 128	1,200	39,321,600	1.609495
Muon regularized neural, batch 128	1,200	39,321,600	1.542515
Muon regularized neural, batch 256	1,200	78,643,200	1.525938
Local QKV neural, batch 256	1,200	78,643,200	1.496975
Wide local neural, batch 512	1,200	157,286,400	1.488248
Wide hybrid, final calibration	1,200	157,286,400	1.421704

Equal processed-target comparison

Neural predictor	Gradient targets	Validation BPB
Baseline at update 1,200	9,830,400	2.071087
Final wide recipe at update 75	9,830,400	2.026129

The same-target control improves BPB by 0.044959. It compares whole recipes: architecture, batch size, optimizer, device and schedule position differ. Update 75 is early in the final 1,200-update cosine schedule, whereas the baseline is at its schedule end. This is evidence of improvement at equal processed targets, not proof that local convolution or Muon alone caused it.



The final run continues to improve slowly after update 600. Its full-budget result must not be described as an equal-compute comparison with the baseline: it uses 16x the gradient targets and a much larger model. Count construction and validation search add further work.

5. Ablation and unsuccessful trials

Remove one inference component without retuning

Each diagnostic starts from the frozen predictor and removes the named component while holding all remaining settings and weights fixed. Only validation is scored. These diagnostics were completed after the model was frozen and were not used to change it. Neural-only retains the already-selected temperature of 1.2.

Variant	Validation BPB	Increase vs full
Full frozen predictor	1.421704	-
Without train-only counts	1.436441	+0.014737
Without continuous cache	1.424677	+0.002973
Without suffix copying	1.426595	+0.004891
Calibrated neural only	1.448975	+0.027271

All three removed components worsen validation BPB. The count expert gives the largest single removal penalty. This supports complementarity between the neural predictor and the compact statistics in this trained model. The penalties are not additive because mixture components interact. They do not replace a matched retraining ablation of local convolution, RoPE, SwiGLU or the optimizer; those separate causal claims are not established here.

What did not help enough

More capacity: a width-448, five-block prototype took 5.337900x baseline CPU time. It was rejected before full training, despite fitting the RAM and size budgets. An earlier local-convolution implementation also exceeded time limits until its CPU computation was optimized.

Hashed input features: a direct-token n-gram augmentation inspired by N-Grammer [6] lowered training loss but gave worse validation performance. Its final calibrated mixture scored 1.510174; same-run averaging gave 1.509272. This is consistent with overfitting in that recipe, not evidence against all n-gram augmentations. An initial CPU/MPS integer-remainder mismatch was fixed with power-of-two bitwise hashing; the failed probe cost is disclosed.

Averaging and retrieval: averaging 25% of wide step 900 with 75% of step 1,200 changed the earlier hybrid score from 1.424376 to 1.424250, still worse than the final global calibration. A separate compact kNN experiment improved an older AdamW hybrid only from about 1.506949 to 1.504584 and required additional indexing and features. Neither method is included in the final model.

A stronger-regularization follow-up was queued but cancelled before training when development was closed. No result is claimed for that proposal.

6. Frozen test and computational cost

The model and inference source hashes were frozen at **2026-09-25 19:25:28 UTC** (26 September, UTC+8). Three alternating baseline/candidate runs then used the unchanged complete-test scorer, CPU FP32 and four threads. All candidate runs gave exactly 1.4381909205827006 BPB; baseline test BPB was 2.1012651958438786. No test-based tuning followed.

Repeat	Baseline scoring (s)	Candidate scoring (s)
1	6.645746	31.053597
2	6.644303	31.480348
3	6.679934	31.049315

Constraint	Measured	Limit	Pass
Ratio of median scoring times	4.672703x	5x	Yes
Maximum candidate RSS	2.580750 GiB	4 GiB	Yes
Uncompressed inference files	58.568878 MiB	64 MiB	Yes

RSS includes process loading and benchmark preparation; timing is the teacher scorer's forward-scoring interval. Inference size includes the final checkpoint and required prediction/evaluation source files, not training tools, logs, supplied benchmark data or the independent baseline checkpoint. The resource result is a same-machine measurement, not a guaranteed ratio on every CPU. The model has limited CPU headroom, so peer verification remains important.

Development cost disclosure

Recorded cost	Amount
Final neural training	8,948.830 s; 157,286,400 gradient targets
All completed recorded gradient runs / probes	580,096,128 targets; 28,366.388 s (7.88 h)
Recorded validation searches	2,100.603 s (35.01 min)
Final count-asset historical build chain	7.410 + 37.203 + 24.209 s (before trimming)

Totals above are sums of recorded process durations, not elapsed wall time or a complete energy estimate. They exclude some scoring/resource checks, packaging, and incomplete runs. Additional disclosed work includes an approximately 450-second aborted CPU run (fewer than 100 updates, exact count unavailable), a failed two-update/64-target hash probe with unrecorded elapsed time, and a kNN index build of about 719 seconds with training-derived feature extraction. JSON evidence retains the separate costs; unrecorded quantities are not treated as zero.

7. Reproducibility, limitations and sources

Reproducing the submitted score

The code and model archives are extracted side by side as MP1_code and MP1_model. Python 3.12, torch 2.7.1, numpy 2.5.3 and tokenizers 0.21.4 are pinned. From MP1_code, install requirements, then run the unchanged evaluator against ../MP1_model/checkpoint.pt with --device cpu --precision fp32 --threads 4 --split test. The README gives complete commands for scoring, verification, resource measurement and optional training. No retraining or external lookup index is required for evaluation.

The frozen checkpoint SHA256 is recorded in FREEZE.json and the archive manifest. Evidence includes complete test JSON records, resource repetitions, equal-target control, search logs and ablations. The clean reconstruction path builds only count orders 1-5 and restores all trigram continuation entries; tensor equality with the historically built-and-trimmed final asset is checked. Rebuilt metadata/file hashes can differ. Stochastic retraining on different devices is not promised to be bitwise identical.

Interpretation and limitations

The submitted predictor improves on the supplied baseline and meets the measured limits, but does not reach the requested 1.4-BPB goal. The main comparison combines more parameters, more training targets and several algorithmic changes. Development uses one main seed and many validation choices, so selection bias and sensitivity to seeds remain unresolved. The test score is a frozen evaluation, not a signal for another search. Component-removal results establish dependence within the final mixture, not isolated architectural superiority.

Substantive AI assistance and attribution

Codex assisted with method proposals, implementation, optimization/debugging, experiment execution, search, evaluation, packaging and drafting this report. Reused algorithmic ideas are cited below; the Muon adaptation retains its MIT license. Teacher code and data attribution are preserved. No pretrained weights or external training text were introduced. This report describes adaptations rather than claiming reproduction of the cited papers.

References

[1] Su et al. RoFormer: Enhanced Transformer with Rotary Position Embedding. 2021. https://arxiv.org/abs/2104.09864

[2] Shazeer. GLU Variants Improve Transformer. 2020. https://arxiv.org/abs/2002.05202

[3] So et al. Primer: Searching for Efficient Transformers for Language Modeling. 2021. https://arxiv.org/abs/2109.08668

[4] Grave, Joulin and Usunier. Improving Neural Language Models with a Continuous Cache. 2016. https://arxiv.org/abs/1612.04426

[5] Keller Jordan and contributors. Muon optimizer implementation (MIT). https://github.com/KellerJordan/Muon

[6] Roy et al. N-Grammer: Augmenting Transformers with latent n-grams. 2022. https://arxiv.org/abs/2207.06366

[7] Supplied DASE7506 MP1 GUIDE, README, evaluator and dataset manifest. Protocol 7506-mp1-wt2-v2. Dataset attribution and licenses: DATA_ATTRIBUTION.md.